

Advanced Terraform, Ansible & Kubernetes Labs

5 Experimental, Graduate-Level Labs: DR, GitOps, Zero-Trust Security, Vault
Secrets & Chaos-Tested Autoscaling

Terraform / Ansible / Kubernetes Lab Series - Advanced Track

Contents

About This Advanced Lab Set	3
Lab 7 — Multi-Region Disaster Recovery (Terraform + Ansible + Velero)	4
Difficulty: Advanced / Experimental	4
Use Case	4
Architecture	4
Directory Layout	4
Prerequisites	4
Step-by-Step Configuration & Execution	5
Validation Steps	6
Cleanup	7
Stretch Goals (for students who finish early)	7
Key Concepts Reinforced	7
Appendix: Full Source Code	8
ansible/roles/velero_setup/tasks/main.yml	8
ansible/site.yml	9
k8s/stateful-app.yaml	9
terraform/dr/main.tf	11
terraform/dr/outputs.tf	12
terraform/dr/variables.tf	12
terraform/primary/main.tf	12
terraform/primary/outputs.tf	14
terraform/primary/variables.tf	15
Lab 8 — GitOps Progressive Delivery (Terraform + Ansible + ArgoCD + Argo Rollouts)	16
Difficulty: Advanced / Experimental	16
Use Case	16
Architecture	16
Directory Layout	16
Prerequisites	16
Step-by-Step Configuration & Execution	17
Validation Steps	18
Cleanup	18
Stretch Goals	18
Key Concepts Reinforced	18
Appendix: Full Source Code	20
ansible/roles/argo_rollouts/tasks/main.yml	20
ansible/roles/argocd/tasks/main.yml	20
ansible/site.yml	21

k8s/argocd-application.yaml	22
k8s/rollout.yaml	22
terraform/main.tf	23
terraform/outputs.tf	24
terraform/variables.tf	24

Lab 9 — Zero-Trust Kubernetes Hardening (CIS + RBAC + NetworkPolicy + OPA Gatekeeper)	25
Difficulty: Advanced / Experimental	25
Use Case	25
Architecture	25
Directory Layout	25
Prerequisites	26
Step-by-Step Configuration & Execution	26
Validation Steps	27
Cleanup	27
Stretch Goals	28
Key Concepts Reinforced	28

Appendix: Full Source Code	29
ansible/roles/cis_hardening/handlers/main.yml	29
ansible/roles/cis_hardening/tasks/main.yml	29
ansible/roles/kube_bench/tasks/main.yml	30
ansible/site.yml	31
k8s/gatekeeper/constraint-no-privileged.yaml	31
k8s/gatekeeper/constraint-require-limits.yaml	31
k8s/gatekeeper/constraint-template-no-privileged.yaml	32
k8s/gatekeeper/constraint-template-require-limits.yaml	32
k8s/gatekeeper/install-note.md	32
k8s/policies/allow-app-to-db.yaml	33
k8s/policies/default-deny-network-policy.yaml	33
k8s/policies/violating-pod-example.yaml	33
k8s/rbac/least-privilege-role.yaml	34
terraform/main.tf	34
terraform/outputs.tf	36
terraform/variables.tf	36

Lab 10 — HashiCorp Vault Dynamic Secrets for Kubernetes	37
Difficulty: Advanced / Experimental	37
Use Case	37
Architecture	37
Directory Layout	37
Prerequisites	38
Step-by-Step Configuration & Execution	38
Validation Steps	39
Cleanup	39
Stretch Goals	39
Key Concepts Reinforced	40

Appendix: Full Source Code	41
ansible/roles/vault_k8s_auth/tasks/main.yml	41
ansible/roles/vault_server/tasks/main.yml	43
ansible/roles/vault_server/templates/vault.hcl.j2	45
ansible/site.yml	45
k8s/sample-app-with-injection.yaml	45
k8s/vault-agent-injector-values.yaml	46

terraform/main.tf	46
terraform/outputs.tf	48
terraform/variables.tf	48
Lab 11 — Autoscaling Under Chaos (HPA + KEDA + Chaos Mesh)	49
Difficulty: Advanced / Experimental	49
Use Case	49
Architecture	49
Directory Layout	49
Prerequisites	50
Step-by-Step Configuration & Execution	50
Validation Steps	51
Cleanup	51
Stretch Goals	51
Key Concepts Reinforced	52
Appendix: Full Source Code	53
ansible/roles/chaos_mesh/tasks/main.yml	53
ansible/roles/keda/tasks/main.yml	53
ansible/roles/loadtest_tools/tasks/main.yml	54
ansible/site.yml	54
k8s/chaos-podkill-experiment.yaml	55
k8s/deployment-and-hpa.yaml	55
k8s/keda-scaledobject.yaml	56
k8s/loadtest.js	57
terraform/main.tf	57
terraform/outputs.tf	58
terraform/variables.tf	58

About This Advanced Lab Set

This set of 5 labs is intentionally tougher and more experimental than the core introductory series (Labs 1-6). Each lab combines Terraform, Ansible, and Kubernetes around a real production concern – disaster recovery, GitOps progressive delivery, zero-trust security hardening, secrets management, and chaos-tested autoscaling – and each one includes a step where students deliberately try to break something (kill the primary region, submit a non-compliant pod, revoke a secret binding, kill pods under load) to prove the control actually works, rather than just deploying it and assuming it does.

Recommended order: these labs assume comfort with the core Labs 1-6 (remote state, workspaces, dynamic blocks, Terraform+Ansible+Docker, and a self-managed kubeadm cluster). Labs 7-11 can then be done in any order, though Lab 9 (Zero-Trust Hardening) pairs well as a prerequisite mindset for Lab 10 (Vault).

Cost and time warning: several of these labs provision larger or multi-region infrastructure (Lab 7 uses two AWS regions; Lab 11 uses a larger instance type) and are designed to run for an extended lab session. Always run each lab’s Cleanup section when finished.

Lab 7 — Multi-Region Disaster Recovery (Terraform + Ansible + Velero)

Difficulty: Advanced / Experimental

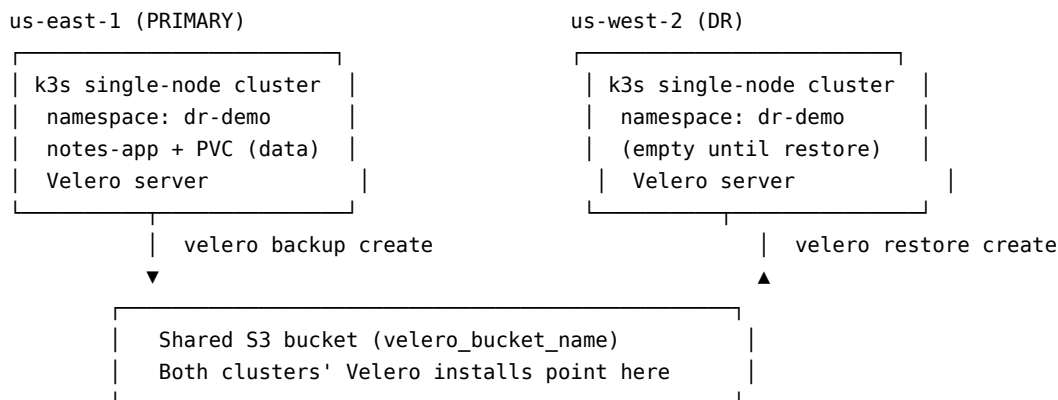
Students should already be comfortable with Labs 1-6 (core Terraform, Terraform+Ansible+Docker, and a self-managed kubernetes cluster) before attempting this lab.

Use Case

A platform team must prove they can actually **recover a Kubernetes workload's data into a different AWS region** after a simulated regional failure — not just describe a DR plan on paper. This lab provisions two independent single-node Kubernetes clusters in two different AWS regions (us-east-1 primary, us-west-2 DR) using Terraform, bootstraps both with Ansible, installs **Velero** (the standard open-source Kubernetes backup/restore tool) pointed at a **shared S3 bucket**, deploys a stateful sample app with real data on a PersistentVolumeClaim, backs it up from the primary region, **deliberately destroys the primary cluster**, and restores the exact same workload and data into the DR cluster.

This is intentionally “tougher” than earlier labs: two independent Terraform state directories, cross-region S3/IAM sharing, and a real destructive failure simulation are all new concepts here.

Architecture



Directory Layout

```

lab7-multiregion-dr-velero/
├── terraform/
│   ├── primary/      # us-east-1: k3s VM + S3 bucket + Velero IAM user (apply FIRST)
│   └── dr/           # us-west-2: k3s VM only, reuses primary's bucket/IAM (apply SECOND)
├── ansible/
│   ├── site.yml
│   └── roles/velero_setup/tasks/main.yml # installs k3s + Velero on whichever host
└── k8s/
    └── stateful-app.yaml # namespace + PVC + Deployment + Service (the workload we protect)
  
```

Prerequisites

- Terraform >= 1.5.0, AWS CLI configured with permissions in **both** us-east-1 and us-west-2
- Ansible >= 2.14
- An EC2 key pair that exists in **both** regions (or one per region)

- `kubectl` locally (optional, for direct verification)

Step-by-Step Configuration & Execution

Step 1 — Provision the PRIMARY region first

```
cd terraform/primary
terraform init
terraform apply \
  -var="key_name=my-ec2-keypair" \
  -var="private_key_path=$HOME/.ssh/my-ec2-keypair.pem" \
  -var="velero_bucket_name=my-org-velero-dr-lab-<unique-suffix>"
```

Note the outputs — you will feed `velero_bucket`, `velero_access_key_id`, and `velero_secret_access_key` into the DR region's Terraform next.

Step 2 — Provision the DR region, reusing the primary's bucket/IAM

```
cd ../dr
terraform init
terraform apply \
  -var="key_name=my-ec2-keypair" \
  -var="private_key_path=$HOME/.ssh/my-ec2-keypair.pem" \
  -var="velero_bucket_name=<same bucket name from Step 1>" \
  -var="velero_access_key_id=$(cd ../primary && terraform output -raw velero_access_key_id)" \
  -var="velero_secret_access_key=$(cd ../primary && terraform output -raw velero_secret_access_key)"
```

Both Terraform runs generate their own Ansible inventory file (`inventory-primary.ini`, `inventory-dr.ini`).

Step 3 — Bootstrap both clusters + Velero with Ansible

```
cd ../../ansible
until ansible -i inventory-primary.ini primary -m ping; do sleep 5; done
until ansible -i inventory-dr.ini dr -m ping; do sleep 5; done

ansible-playbook -i inventory-primary.ini -i inventory-dr.ini site.yml
```

Step 4 — Deploy the sample stateful app on the PRIMARY cluster

```
ssh -i ~/.ssh/my-ec2-keypair.pem ubuntu@<primary_public_ip>
export KUBECONFIG=/home/ubuntu/.kube/config
k3s kubectl apply -f - <<'MANIFEST'
# paste the contents of k8s/stateful-app.yaml here, or scp the file first:
#   scp -i key.pem k8s/stateful-app.yaml ubuntu@<primary_public_ip>:/home/ubuntu/
MANIFEST
```

(Simplest in practice: `scp` the manifest up, then `k3s kubectl apply -f stateful-app.yaml`.)

Confirm data was written:

```
k3s kubectl exec -n dr-demo deploy/notes-app -- cat /data/notes.txt
```

Step 5 — Take a Velero backup on the PRIMARY cluster

```
velero backup create dr-demo-backup-01 --include-namespaces dr-demo --wait
velero backup describe dr-demo-backup-01
```

Step 6 — Simulate a regional failure

```
cd terraform/primary
terraform destroy \
  -var="key_name=my-ec2-keypair" \
  -var="private_key_path=$HOME/.ssh/my-ec2-keypair.pem" \
  -var="velero_bucket_name=<bucket-name>"
```

This destroys the primary EC2 instance — the “region is gone” simulation — while leaving the S3 bucket (backups) intact since it lives in a separate resource lifecycle from what you just destroyed... **unless** you also destroy the bucket. Do not tear down the S3 bucket in this step.

Step 7 — Restore into the DR cluster

```
ssh -i ~/.ssh/my-ec2-keypair.pem ubuntu@<dr_public_ip>
export KUBECONFIG=/home/ubuntu/.kube/config
velero backup get                # confirm the backup is visible from the DR cluster too
velero restore create --from-backup dr-demo-backup-01 --wait
```

Validation Steps

1. Confirm the backup exists in shared storage from BOTH clusters’ perspective

```
aws s3 ls s3://<velero_bucket_name>/backups/ --recursive
```

2. Confirm the restore recreated the namespace, PVC, and Deployment on DR

```
k3s kubectl get ns dr-demo
k3s kubectl get pvc,deploy,pod -n dr-demo
```

3. Confirm the DATA itself survived the restore (the actual DR test)

```
k3s kubectl exec -n dr-demo deploy/notes-app -- cat /data/notes.txt
```

The timestamp written in Step 4 (on the now-destroyed primary cluster) must still be present — this is proof the underlying EBS volume snapshot, not just the Kubernetes object definitions, was restored.

4. Confirm restore status is Completed with no errors

```
velero restore describe --from-backup dr-demo-backup-01 <restore-name>
```

5. RTO/RPO discussion (write this down as part of the lab report):

time how long Steps 6-7 took end-to-end. That elapsed time is your measured Recovery Time Objective (RTO) for this workload with this tooling. The Recovery Point Objective (RPO) is however old your last velero backup create was relative to the failure — discuss how scheduled backups (velero schedule create) would improve this.

Cleanup

```
cd terraform/dr    && terraform destroy -var="key_name=..." -var="private_key_path=..."
-var="velero_bucket_name=..." -var="velero_access_key_id=..." -var="velero_secret_access_key=..."
cd ../primary      # already destroyed in Step 6, but confirm:
terraform destroy -var="key_name=..." -var="private_key_path=..." -var="velero_bucket_name=..."
aws s3 rb s3://<velero_bucket_name> --force # only after confirming you no longer need the backups
```

Stretch Goals (for students who finish early)

- Add velero schedule create daily-backup --schedule="0 * * * *" --include-namespaces dr-demo and measure how it changes your RPO discussion.
- Break the restore on purpose (delete the S3 object between backup and restore) and document the failure mode.
- Repeat the lab using an EBS-CSI-backed StorageClass instead of the default k3s local-path provisioner, and compare restore behavior.

Key Concepts Reinforced

- Structuring Terraform as **two independent state directories** that must be applied in a specific order and share output values via -var — a common real-world pattern for multi-region/multi-account setups
- Velero's separation of **backup storage location** (S3) from **snapshot location** (EBS) and why both matter for full data recovery
- The difference between restoring Kubernetes **object definitions** (easy) and restoring the **underlying data** (the actual hard part of DR)
- Measuring RTO/RPO empirically instead of just defining them in a document

Appendix: Full Source Code

Every configuration and code file for this lab is listed below in full, organized by path, so the lab can be reproduced without needing the original repository.

ansible/roles/velero_setup/tasks/main.yml

```
---
- name: Install k3s (lightweight single-node Kubernetes)
  shell: curl -sL https://get.k3s.io | sh -
  args:
    creates: /usr/local/bin/k3s

- name: Wait for k3s node to be Ready
  shell: k3s kubectl get nodes | grep -q Ready
  register: k3s_ready
  until: k3s_ready.rc == 0
  retries: 12
  delay: 10
  changed_when: false

- name: Make kubeconfig readable and export KUBECONFIG for this user
  shell: |
    mkdir -p /home/ubuntu/.kube
    cp /etc/rancher/k3s/k3s.yaml /home/ubuntu/.kube/config
    sed -i 's/127.0.0.1/{{ ansible_default_ipv4.address }}/' /home/ubuntu/.kube/config
    chown ubuntu:ubuntu /home/ubuntu/.kube/config
  args:
    creates: /home/ubuntu/.kube/config

- name: Download the Velero CLI
  unarchive:
    src:
      "https://github.com/vmware-tanzu/velero/releases/download/v1.14.0/velero-v1.14.0-linux-amd64.tar.gz"
    dest: /tmp
    remote_src: true
    creates: /tmp/velero-v1.14.0-linux-amd64

- name: Install the Velero CLI binary
  copy:
    src: /tmp/velero-v1.14.0-linux-amd64/velero
    dest: /usr/local/bin/velero
    mode: "0755"
    remote_src: true

- name: Write the AWS credentials file Velero's AWS plugin expects
  copy:
    dest: /home/ubuntu/velero-credentials
    mode: "0600"
    owner: ubuntu
    content: |
      [default]
      aws_access_key_id={{ aws_access_key_id }}
      aws_secret_access_key={{ aws_secret_access_key }}
```

```

- name: Check whether Velero is already installed in this cluster
  become_user: ubuntu
  environment:
    KUBECONFIG: /home/ubuntu/.kube/config
  command: k3s kubectl get deployment velero -n velero
  register: velero_check
  failed_when: false
  changed_when: false

- name: Install Velero server into the cluster (AWS plugin, shared bucket)
  become_user: ubuntu
  environment:
    KUBECONFIG: /home/ubuntu/.kube/config
  command: >
    velero install
    --provider aws
    --plugins velero/velero-plugin-for-aws:v1.10.0
    --bucket {{ velero_bucket }}
    --backup-location-config region={{ aws_region }}
    --snapshot-location-config region={{ aws_region }}
    --secret-file /home/ubuntu/velero-credentials
    --use-node-agent
    --wait
  when: velero_check.rc != 0

```

ansible/site.yml

```

---
# Bootstraps a lightweight single-node k3s cluster on BOTH primary and dr
# hosts (k3s is used instead of full kubeadm to keep this lab focused on
# the Velero backup/restore workflow rather than repeating cluster
# bootstrap mechanics already covered in the kubeadm lab), then installs
# Velero against each cluster pointing at the SAME shared S3 bucket.

- name: Bootstrap k3s + Velero on the primary cluster
  hosts: primary
  become: true
  roles:
    - velero_setup

- name: Bootstrap k3s + Velero on the DR cluster
  hosts: dr
  become: true
  roles:
    - velero_setup

```

k8s/stateful-app.yml

```

apiVersion: v1
kind: Namespace
metadata:
  name: dr-demo
---

```

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: notes-pvc
  namespace: dr-demo
spec:
  accessModes: ["ReadWriteOnce"]
  resources:
    requests:
      storage: 1Gi
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: notes-app
  namespace: dr-demo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: notes-app
  template:
    metadata:
      labels:
        app: notes-app
    spec:
      containers:
        - name: notes-app
          image: busybox
          command:
            - sh
            - -c
            - "echo 'DR-DEMO: data written at ' $(date) >> /data/notes.txt; sleep infinity"
          volumeMounts:
            - name: notes-storage
              mountPath: /data
      volumes:
        - name: notes-storage
          persistentVolumeClaim:
            claimName: notes-pvc
---
apiVersion: v1
kind: Service
metadata:
  name: notes-app-svc
  namespace: dr-demo
spec:
  selector:
    app: notes-app
  ports:
    - port: 80
      targetPort: 80
```

terraform/dr/main.tf

```

terraform {
  required_version = ">= 1.5.0"
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
    local = { source = "hashicorp/local", version = "~> 2.4" }
  }
}

provider "aws" {
  region = "us-west-2"
}

# NOTE: the DR cluster does NOT create its own S3 bucket or IAM user -
# it reuses the SAME velero_bucket_name and IAM keys created by the
# primary region's Terraform (passed in as variables), because Velero
# restore requires reading from the exact bucket the backup was written to.

data "aws_vpc" "default" {
  default = true
}

data "aws_subnets" "default" {
  filter {
    name = "vpc-id"
    values = [data.aws_vpc.default.id]
  }
}

data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]
  filter {
    name = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"]
  }
}

resource "aws_security_group" "cluster" {
  name_prefix = "lab7-dr-sg-"
  vpc_id      = data.aws_vpc.default.id

  ingress { description = "SSH"      from_port = 22 to_port = 22 protocol = "tcp" cidr_blocks = ["0.0.0.0/0"] }
  ingress { description = "K8s API" from_port = 6443 to_port = 6443 protocol = "tcp" cidr_blocks = ["0.0.0.0/0"] }
  ingress { description = "NodePort" from_port = 30000 to_port = 32767 protocol = "tcp" cidr_blocks = ["0.0.0.0/0"] }
  ingress { description = "Intra-cluster" from_port = 0 to_port = 0 protocol = "-1" self = true }
  egress { from_port = 0 to_port = 0 protocol = "-1" cidr_blocks = ["0.0.0.0/0"] }

  tags = { Name = "lab7-dr-sg" }
}

resource "aws_instance" "dr_node" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = var.instance_type
  subnet_id     = data.aws_subnets.default.ids[0]

```

```

    vpc_security_group_ids = [aws_security_group.cluster.id]
    key_name                = var.key_name
    root_block_device { volume_size = 20 }
    tags = { Name = "lab7-dr-k8s-node" }
  }

resource "local_file" "dr_inventory" {
  filename = "${path.module}/../../ansible/inventory-dr.ini"
  content  = <<-EOT
[dr]
${aws_instance.dr_node.public_ip} ansible_user=ubuntu ansible_ssh_private_key_file=${var.private_key_path} ansible_ssh
o StrictHostKeyChecking=no' private_ip=${aws_instance.dr_node.private_ip}

[dr:vars]
velero_bucket=${var.velero_bucket_name}
aws_access_key_id=${var.velero_access_key_id}
aws_secret_access_key=${var.velero_secret_access_key}
aws_region=us-west-2
EOT
}

```

terraform/dr/outputs.tf

```

output "dr_public_ip" {
  value = aws_instance.dr_node.public_ip
}

```

terraform/dr/variables.tf

```

variable "instance_type"          { default = "t3.medium" }
variable "key_name"               { type = string }
variable "private_key_path"      { type = string }
variable "velero_bucket_name"    { type = string }
variable "velero_access_key_id"  { type = string }
variable "velero_secret_access_key" {
  type      = string
  sensitive = true
}

```

terraform/primary/main.tf

```

terraform {
  required_version = ">= 1.5.0"
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
    local = { source = "hashicorp/local", version = "~> 2.4" }
  }
}

provider "aws" {
  region = "us-east-1"
}

```

S3 bucket used by Velero as the backup storage location.
 # Both clusters (primary + DR) point at this SAME bucket so a backup

```
# taken in the primary region can be restored into the DR region.
resource "aws_s3_bucket" "velero_backups" {
  bucket          = var.velero_bucket_name
  force_destroy   = true
}

resource "aws_s3_bucket_versioning" "velero_backups" {
  bucket = aws_s3_bucket.velero_backups.id
  versioning_configuration { status = "Enabled" }
}

# IAM user Velero uses to read/write the backup bucket + take EBS snapshots
resource "aws_iam_user" "velero" {
  name = "velero-backup-user-${var.velero_bucket_name}"
}

resource "aws_iam_access_key" "velero" {
  user = aws_iam_user.velero.name
}

resource "aws_iam_user_policy" "velero" {
  name = "velero-policy"
  user = aws_iam_user.velero.name
  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Effect   = "Allow"
        Action   = ["ec2:DescribeVolumes", "ec2:DescribeSnapshots", "ec2:CreateTags",
                  "ec2:CreateVolume", "ec2:CreateSnapshot", "ec2:DeleteSnapshot"]
        Resource = "*"
      },
      {
        Effect   = "Allow"
        Action   = ["s3:GetObject", "s3:DeleteObject", "s3:PutObject", "s3:AbortMultipartUpload",
                  "s3:ListMultipartUploadParts"]
        Resource = "${aws_s3_bucket.velero_backups.arn}/*"
      },
      {
        Effect   = "Allow"
        Action   = ["s3:ListBucket"]
        Resource = aws_s3_bucket.velero_backups.arn
      }
    ]
  })
}

data "aws_vpc" "default" {
  default = true
}

data "aws_subnets" "default" {
  filter {
    name     = "vpc-id"
    values   = [data.aws_vpc.default.id]
  }
}
```

```
}
```

```
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]
  filter {
    name     = "name"
    values   = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"]
  }
}
```

```
resource "aws_security_group" "cluster" {
  name_prefix = "lab7-primary-sg-"
  vpc_id      = data.aws_vpc.default.id

  ingress { description = "SSH"      from_port = 22 to_port = 22 protocol = "tcp" cidr_blocks = ["0.0.0.0/0"] }
  ingress { description = "K8s API"  from_port = 6443 to_port = 6443 protocol = "tcp" cidr_blocks = ["0.0.0.0/0"] }
  ingress { description = "NodePort" from_port = 30000 to_port = 32767 protocol = "tcp" cidr_blocks = ["0.0.0.0/0"] }
  ingress { description = "Intra-cluster" from_port = 0 to_port = 0 protocol = "-1" self = true }
  egress { from_port = 0 to_port = 0 protocol = "-1" cidr_blocks = ["0.0.0.0/0"] }

  tags = { Name = "lab7-primary-sg" }
}
```

```
resource "aws_instance" "primary_node" {
  ami                = data.aws_ami.ubuntu.id
  instance_type      = var.instance_type
  subnet_id          = data.aws_subnets.default.ids[0]
  vpc_security_group_ids = [aws_security_group.cluster.id]
  key_name            = var.key_name
  root_block_device { volume_size = 20 }
  tags = { Name = "lab7-primary-k8s-node" }
}
```

```
resource "local_file" "primary_inventory" {
  filename = "${path.module}/../../ansible/inventory-primary.ini"
  content  = <<-EOT
[primary]
${aws_instance.primary_node.public_ip} ansible_user=ubuntu ansible_ssh_private_key_file=${var.private_key_path} ansible_host=${aws_instance.primary_node.public_ip}
o StrictHostKeyChecking=no' private_ip=${aws_instance.primary_node.private_ip}

[primary:vars]
velero_bucket=${var.velero_bucket_name}
aws_access_key_id=${aws_iam_access_key.velero.id}
aws_secret_access_key=${aws_iam_access_key.velero.secret}
aws_region=us-east-1
EOT
}
```

terraform/primary/outputs.tf

```
output "primary_public_ip" {
  value = aws_instance.primary_node.public_ip
}

output "velero_bucket" {
  value = aws_s3_bucket.velero_backups.bucket
}
```

```
}  
output "velero_access_key_id" {  
    value = aws_iam_access_key.velero.id  
}  
output "velero_secret_access_key" {  
    value      = aws_iam_access_key.velero.secret  
    sensitive = true  
}
```

terraform/primary/variables.tf

```
variable "instance_type"    { default = "t3.medium" }  
variable "key_name"         { type = string }  
variable "private_key_path" { type = string }  
variable "velero_bucket_name" { type = string }
```

Lab 8 — GitOps Progressive Delivery (Terraform + Ansible + ArgoCD + Argo Rollouts)

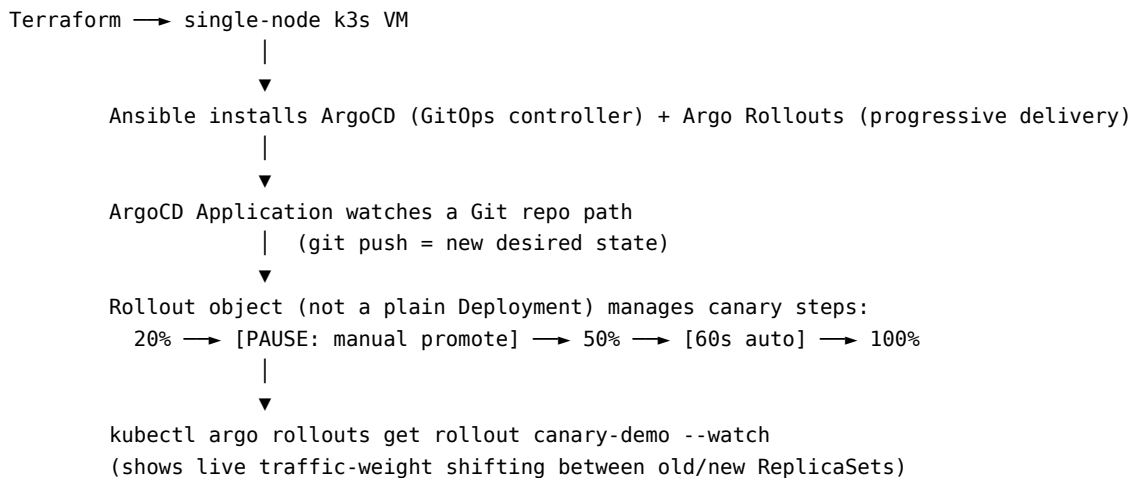
Difficulty: Advanced / Experimental

Builds on Labs 1-6. Assumes comfort with basic Kubernetes objects (Deployment, Service) and a conceptual understanding of CI/CD.

Use Case

A platform team wants **deployments that ship themselves from Git** (GitOps) instead of being pushed by a CI pipeline, and wants **canary rollouts with a manual approval gate** instead of all-or-nothing `kubectl apply`. This lab provisions a single-node k3s cluster with Terraform, uses Ansible to install **ArgoCD** (the GitOps controller that continuously reconciles cluster state to match a Git repo) and **Argo Rollouts** (a progressive-delivery controller that replaces the native Deployment object with canary/blue-green traffic shifting), then walks through a real canary release with a pause gate that must be manually promoted.

Architecture



Directory Layout

```

lab8-gitops-argo-rollouts/
├── terraform/
│   ├── main.tf          # single k3s VM + SG (22, 6443, 30443 ArgoCD UI, NodePort range)
│   ├── variables.tf
│   └── outputs.tf
├── ansible/
│   ├── site.yml
│   └── roles/
│       ├── argocd/tasks/main.yml      # k3s + ArgoCD install + NodePort expose + admin password
│       └── argo_rollouts/tasks/main.yml # Argo Rollouts controller + kubectl plugin
└── k8s/
    ├── rollout.yaml          # the canary Rollout + Service
    └── argocd-application.yaml # example ArgoCD Application pointing at a Git path
  
```

Prerequisites

- Terraform $\geq$ 1.5.0, AWS CLI configured

- Ansible $\geq$ 2.14
- An EC2 key pair
- A GitHub account (to fork this lab's manifests for the GitOps portion)
- kubectl locally (optional)

Step-by-Step Configuration & Execution

Step 1 — Provision the cluster VM

```
cd terraform
terraform init
terraform apply -var="key_name=my-ec2-keypair" -var="private_key_path=$HOME/.ssh/my-ec2-keypair.pem"
```

Step 2 — Bootstrap k3s, ArgoCD, and Argo Rollouts

```
cd ../ansible
until ansible -i inventory.ini gitops -m ping; do sleep 5; done
ansible-playbook -i inventory.ini site.yml
```

Save the ArgoCD admin password printed in the debug task output.

Step 3 — Log into the ArgoCD UI

`https://<gitops_node_public_ip>:30443`

Username: admin, password: from Step 2. Accept the self-signed cert warning.

Step 4 — Fork this lab into your own Git repo

GitOps requires ArgoCD to watch a **real Git repository**, not a local file. Fork/push this lab's k8s/ folder to your own GitHub repo, then edit k8s/argocd-application.yaml to point repoURL/path at your fork.

Step 5 — Register the Application with ArgoCD

```
ssh -i ~/.ssh/my-ec2-keypair.pem ubuntu@<gitops_node_public_ip>
export KUBECONFIG=/home/ubuntu/.kube/config
k3s kubectl apply -f argocd-application.yaml # scp this file up first
argocd app sync canary-demo-app
```

Step 6 — Watch the canary rollout live

```
kubectl argo rollouts get rollout canary-demo --watch
```

It will progress to 20% and then **stop at the manual pause step**.

Step 7 — Promote the canary manually

```
kubectl argo rollouts promote canary-demo
```

Watch it shift to 50%, auto-wait 60 seconds, then continue to 100%.

Step 8 — Trigger a NEW rollout via GitOps (the actual point of the lab)

Change the image in your forked repo's `k8s/rollout.yaml` from `argoproj/rollouts-demo:blue` to `argoproj/rollouts-demo:yellow`, commit, and push. **Do not run `kubectl apply` yourself.** Instead:

```
argocd app sync canary-demo-app      # or wait for ArgoCD's auto-sync loop
kubectl argo rollouts get rollout canary-demo --watch
```

You should see a brand-new canary rollout begin automatically, driven purely by the Git push — this is the GitOps loop working end-to-end.

Validation Steps

1. ArgoCD shows the app as Synced and Healthy

```
argocd app get canary-demo-app
```

2. The Rollout actually paused at the manual gate (did not skip past 20%)

```
kubectl argo rollouts status canary-demo
```

3. Traffic weight matches the canary step — confirm two ReplicaSets coexist during the canary window with the expected pod-count ratio:

```
kubectl get rs -l app=canary-demo
```

4. A Git push alone (no `kubectl apply`) triggered a new rollout — this is the core GitOps assertion. Confirm by checking ArgoCD's sync history:

```
argocd app history canary-demo-app
```

5. Rollback works

```
kubectl argo rollouts undo canary-demo
kubectl argo rollouts get rollout canary-demo --watch
```

Cleanup

```
cd terraform
terraform destroy -var="key_name=my-ec2-keypair" -var="private_key_path=$HOME/.ssh/my-ec2-keypair.pem"
```

Stretch Goals

- Replace the manual pause: `{}` gate with an **AnalysisTemplate** that automatically queries Prometheus and aborts the rollout if error rate exceeds a threshold — true automated canary analysis instead of a human gate.
- Add a second ArgoCD Application in a different namespace to practice the “app of apps” pattern.
- Intentionally push a broken image tag and practice using `kubectl argo rollouts abort` before it reaches 100%.

Key Concepts Reinforced

- The difference between **CI-driven push deployment** and **GitOps pull-based reconciliation**
- Why a `Rollout` object (Argo Rollouts CRD) is needed instead of a plain `Deployment` to get canary/blue-green traffic shifting

- Manual approval gates vs. automated, metric-driven promotion in progressive delivery
- How to prove a deployment was actually driven by Git, not by an operator running `kubectl apply`

Appendix: Full Source Code

Every configuration and code file for this lab is listed below in full, organized by path, so the lab can be reproduced without needing the original repository.

ansible/roles/argo_rollouts/tasks/main.yml

```
---
- name: Create argo-rollouts namespace
  become_user: ubuntu
  environment: { KUBECONFIG: /home/ubuntu/.kube/config }
  command: k3s kubectl create namespace argo-rollouts
  register: ns_create
  failed_when: "ns_create.rc != 0 and 'AlreadyExists' not in ns_create.stderr"
  changed_when: "ns_create.rc == 0"

- name: Install the Argo Rollouts controller
  become_user: ubuntu
  environment: { KUBECONFIG: /home/ubuntu/.kube/config }
  command: >
    k3s kubectl apply -n argo-rollouts -f
    https://github.com/argoproj/argo-rollouts/releases/download/v1.7.0/install.yaml

- name: Install the kubectl-argo-rollouts plugin (for `kubectl argo rollouts` commands)
  get_url:
    url:
      https://github.com/argoproj/argo-rollouts/releases/download/v1.7.0/kubectl-argo-rollouts-linux-amd64
    dest: /usr/local/bin/kubectl-argo-rollouts
    mode: "0755"
```

ansible/roles/argocd/tasks/main.yml

```
---
- name: Install k3s
  shell: curl -sL https://get.k3s.io | sh -
  args:
    creates: /usr/local/bin/k3s

- name: Wait for node Ready
  shell: k3s kubectl get nodes | grep -q Ready
  register: k3s_ready
  until: k3s_ready.rc == 0
  retries: 12
  delay: 10
  changed_when: false

- name: Set up kubeconfig for ubuntu user
  shell: |
    mkdir -p /home/ubuntu/.kube
    cp /etc/rancher/k3s/k3s.yaml /home/ubuntu/.kube/config
    chown ubuntu:ubuntu /home/ubuntu/.kube/config
  args:
    creates: /home/ubuntu/.kube/config
```

```

- name: Create argocd namespace
  become_user: ubuntu
  environment: { KUBECONFIG: /home/ubuntu/.kube/config }
  command: k3s kubectl create namespace argocd
  register: ns_create
  failed_when: "ns_create.rc != 0 and 'AlreadyExists' not in ns_create.stderr"
  changed_when: "ns_create.rc == 0"

- name: Install ArgoCD manifests
  become_user: ubuntu
  environment: { KUBECONFIG: /home/ubuntu/.kube/config }
  command: >
    k3s kubectl apply -n argocd -f
    https://raw.githubusercontent.com/argoproj/argo-cd/v2.11.0/manifests/install.yaml

- name: Wait for ArgoCD server deployment to be available
  become_user: ubuntu
  environment: { KUBECONFIG: /home/ubuntu/.kube/config }
  command: k3s kubectl -n argocd rollout status deployment/argocd-server --timeout=300s
  changed_when: false

- name: Expose ArgoCD UI via NodePort for lab access
  become_user: ubuntu
  environment: { KUBECONFIG: /home/ubuntu/.kube/config }
  command: >
    k3s kubectl -n argocd patch svc argocd-server -p
    '{"spec":{"type":"NodePort","ports":[{"port":443,"targetPort":8080,"nodePort":30443}]}}'

- name: Retrieve the initial ArgoCD admin password
  become_user: ubuntu
  environment: { KUBECONFIG: /home/ubuntu/.kube/config }
  shell: k3s kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" |
base64 -d
  register: argocd_pw
  changed_when: false

- name: Show the ArgoCD admin password (save this!)
  debug:
    msg: "ArgoCD admin password: {{ argocd_pw.stdout }}"

- name: Install the ArgoCD CLI
  get_url:
    url: https://github.com/argoproj/argo-cd/releases/download/v2.11.0/argocd-linux-amd64
    dest: /usr/local/bin/argocd
    mode: "0755"

```

ansible/site.yml

```

---
- name: Bootstrap k3s + ArgoCD + Argo Rollouts for GitOps progressive delivery
  hosts: gitops
  become: true
  roles:
    - argocd

```

```
- argo_rollouts
```

k8s/argocd-application.yaml

```
# Example ArgoCD Application pointing at a Git repo path containing the
# Rollout manifest above. Fork this lab's repo (or any repo) and adjust
# repoURL/path/targetRevision to match your fork before applying.
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: canary-demo-app
  namespace: argocd
spec:
  project: default
  source:
    repoURL: "https://github.com/<your-username>/<your-fork>.git"
    targetRevision: main
    path: lab8-gitops-argo-rollouts/k8s
  destination:
    server: "https://kubernetes.default.svc"
    namespace: default
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

k8s/rollout.yaml

```
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: canary-demo
  namespace: default
spec:
  replicas: 5
  strategy:
    canary:
      steps:
        - setWeight: 20
        - pause: {} # manual gate: hold at 20% until promoted
        - setWeight: 50
        - pause: { duration: 60 } # automatic: hold 60s at 50%, then continue
        - setWeight: 100
  selector:
    matchLabels:
      app: canary-demo
  template:
    metadata:
      labels:
        app: canary-demo
    spec:
      containers:
        - name: canary-demo
```

```

        image: argoproj/rollouts-demo:blue
        ports:
          - containerPort: 8080
        resources:
          requests: { cpu: "20m", memory: "32Mi" }
---
apiVersion: v1
kind: Service
metadata:
  name: canary-demo-svc
  namespace: default
spec:
  selector:
    app: canary-demo
  ports:
    - port: 80
      targetPort: 8080
  type: NodePort

```

terraform/main.tf

```

terraform {
  required_version = ">= 1.5.0"
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
    local = { source = "hashicorp/local", version = "~> 2.4" }
  }
}

provider "aws" {
  region = var.aws_region
}

data "aws_vpc" "default" { default = true }

data "aws_subnets" "default" {
  filter {
    name = "vpc-id"
    values = [data.aws_vpc.default.id]
  }
}

data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]
  filter {
    name = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"]
  }
}

resource "aws_security_group" "gitops" {
  name_prefix = "lab8-gitops-sg-"
  vpc_id      = data.aws_vpc.default.id
}

```

```

    ingress { description = "SSH"      from_port = 22   to_port = 22   protocol = "tcp" cidr_blocks = ["0.0.0.0/0"] }
    ingress { description = "K8s API"  from_port = 6443 to_port = 6443 protocol = "tcp" cidr_blocks = ["0.0.0.0/0"] }
    ingress { description = "ArgoCD UI" from_port = 30443 to_port = 30443 protocol = "tcp" cidr_blocks = ["0.0.0.0/0"] }
    ingress { description = "NodePort" from_port = 30000 to_port = 32767 protocol = "tcp" cidr_blocks = ["0.0.0.0/0"] }
    egress   { from_port = 0 to_port = 0 protocol = "-1" cidr_blocks = ["0.0.0.0/0"] }

    tags = { Name = "lab8-gitops-sg" }
}

resource "aws_instance" "gitops_node" {
    ami                = data.aws_ami.ubuntu.id
    instance_type      = var.instance_type
    subnet_id          = data.aws_subnets.default.ids[0]
    vpc_security_group_ids = [aws_security_group.gitops.id]
    key_name            = var.key_name
    root_block_device { volume_size = 20 }
    tags = { Name = "lab8-gitops-node" }
}

resource "local_file" "inventory" {
    filename = "${path.module}/../ansible/inventory.ini"
    content  = <<-EOT
[gitops]
${aws_instance.gitops_node.public_ip} ansible_user=ubuntu ansible_ssh_private_key_file=${var.private_key_path} ansible_
o StrictHostKeyChecking=no'
EOT
}

```

terraform/outputs.tf

```

output "gitops_node_public_ip" {
    value = aws_instance.gitops_node.public_ip
}

```

terraform/variables.tf

```

variable "aws_region"      { default = "us-east-1" }
variable "instance_type"   { default = "t3.medium" }
variable "key_name"        { type = string }
variable "private_key_path" { type = string }

```

Lab 9 — Zero-Trust Kubernetes Hardening (CIS + RBAC + NetworkPolicy + OPA Gatekeeper)

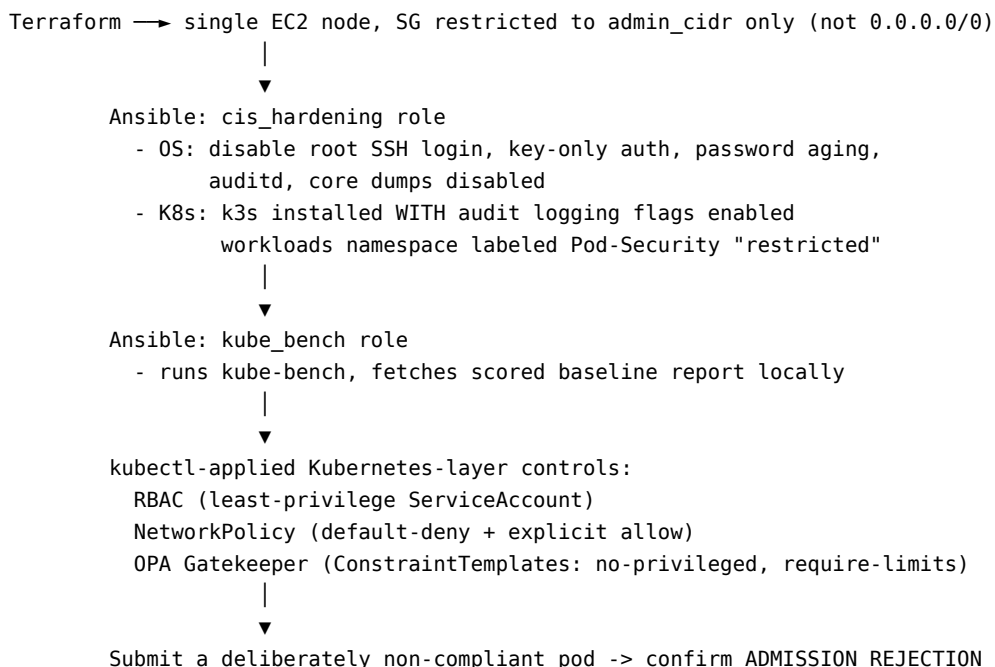
Difficulty: Advanced / Experimental

Builds on Labs 1-6 and assumes familiarity with basic RBAC/NetworkPolicy concepts. This lab is security-first: several steps intentionally try to **break** things to prove a control works, not just to deploy successfully.

Use Case

A security team must prove — not just assert — that a cluster meets a baseline “zero trust” posture: OS-level CIS hardening, least-privilege RBAC instead of broad default permissions, default-deny networking with explicit allow-lists, and admission-time policy enforcement that actually **rejects** non-compliant workloads rather than just flagging them later. This lab provisions a single hardened node with Terraform, applies a representative set of CIS Ubuntu Benchmark controls and Kubernetes audit logging with Ansible, runs kube-bench to produce a scored baseline report, then layers RBAC, NetworkPolicy, and OPA Gatekeeper on top — finishing by deliberately submitting a non-compliant pod and confirming it gets rejected at admission time.

Architecture



Directory Layout

```

lab9-zero-trust-hardening/
├─ terraform/                # single node, SG locked to your IP (admin_cidr)
├─ ansible/
│  ├─ site.yml
│  ├─ generated/kube-bench-baseline.txt # fetched report lands here
│  └─ roles/
│     ├─ cis_hardening/tasks/main.yml # OS + k3s hardening
│     └─ kube_bench/tasks/main.yml    # scored CIS baseline report
└─ k8s/
  
```

```
├─ rbac/least-privilege-role.yaml
├─ policies/default-deny-network-policy.yaml
├─ policies/allow-app-to-db.yaml
├─ policies/violating-pod-example.yaml    # should be REJECTED
├─ gatekeeper/
│   ├── install-note.md
│   ├── constraint-template-no-privileged.yaml
│   ├── constraint-no-privileged.yaml
│   ├── constraint-template-require-limits.yaml
│   └── constraint-require-limits.yaml
```

Prerequisites

- Terraform >= 1.5.0, AWS CLI configured
- Ansible >= 2.14
- An EC2 key pair
- Your own current public IP (`curl -s https://checkip.amazonaws.com`) for `admin_cidr`

Step-by-Step Configuration & Execution

Step 1 — Provision the node with SSH/API locked to YOUR IP only

```
cd terraform
terraform init
terraform apply \
  -var="key_name=my-ec2-keypair" \
  -var="private_key_path=$HOME/.ssh/my-ec2-keypair.pem" \
  -var="admin_cidr=$(curl -s https://checkip.amazonaws.com)/32"
```

Step 2 — Apply CIS hardening + bootstrap k3s with audit logging

```
cd ../ansible
until ansible -i inventory.ini hardened -m ping; do sleep 5; done
ansible-playbook -i inventory.ini site.yml
```

This produces `ansible/generated/kube-bench-baseline.txt` — open it and note the PASS/FAIL/WARN counts before continuing. This is your **before** snapshot.

Step 3 — Apply the Kubernetes-layer RBAC and NetworkPolicy controls

```
ssh -i ~/.ssh/my-ec2-keypair.pem ubuntu@<hardened_node_public_ip>
export KUBECONFIG=/home/ubuntu/.kube/config
# scp the k8s/ folder up first, then:
k3s kubectl apply -f rbac/least-privilege-role.yaml
k3s kubectl apply -f policies/default-deny-network-policy.yaml
k3s kubectl apply -f policies/allow-app-to-db.yaml
```

Step 4 — Install OPA Gatekeeper and apply the constraint templates

```
kubectl apply -f
https://raw.githubusercontent.com/open-policy-agent/gatekeeper/v3.16.0/deploy/gatekeeper.yaml
kubectl -n gatekeeper-system rollout status deployment/gatekeeper-controller-manager --timeout=180s
```

```
k3s kubectl apply -f gatekeeper/constraint-template-no-privileged.yaml
k3s kubectl apply -f gatekeeper/constraint-no-privileged.yaml
k3s kubectl apply -f gatekeeper/constraint-template-require-limits.yaml
k3s kubectl apply -f gatekeeper/constraint-require-limits.yaml
```

Step 5 — Try to break it: submit the non-compliant pod

```
k3s kubectl apply -f policies/violating-pod-example.yaml
```

Expected result: this command should FAIL with an admission-webhook error naming both violations (privileged container, missing resource limits). If it succeeds instead, a constraint isn't wired up correctly — that's a real finding, not a lab bug; go back and check Step 4.

Validation Steps

1. kube-bench baseline was actually produced and shows a real score

```
cat ansible/generated/kube-bench-baseline.txt | grep -E "PASS|FAIL|WARN" | sort | uniq -c
```

2. SSH hardening took effect — confirm password auth is really off:

```
ssh -o PreferredAuthentications=password -o PubkeyAuthentication=no ubuntu@<node_ip>
# Expected: Permission denied (publickey) -- password auth must not work
```

3. NetworkPolicy default-deny is enforced — deploy two test pods (frontend unlabeled vs. app=frontend) and confirm only the correctly-labeled one can reach a backend-db-labeled pod on 5432; the unlabeled one must time out.

4. RBAC least-privilege works — using the app-readonly-sa token, confirm reads succeed but writes are denied:

```
kubectl auth can-i create pods --as=system:serviceaccount:workloads:app-readonly-sa -n workloads
# Expected: no
kubectl auth can-i get pods --as=system:serviceaccount:workloads:app-readonly-sa -n workloads
# Expected: yes
```

5. Gatekeeper actually rejects at admission time (Step 5) — confirm the specific error messages from both ConstraintTemplates appear, not a generic failure.

6. Audit logging is capturing real events

```
sudo tail -20 /var/log/k3s-audit.log
```

Cleanup

```
cd terraform
terraform destroy \
  -var="key_name=my-ec2-keypair" \
  -var="private_key_path=$HOME/.ssh/my-ec2-keypair.pem" \
  -var="admin_cidr=<same value used in Step 1>"
```

Stretch Goals

- Re-run kube-bench after Step 4-5 and diff the report against the Step 2 baseline — quantify how much the score improved.
- Add a Gatekeeper constraint requiring `readOnlyRootFilesystem: true` and find (or write) a pod that violates only that rule.
- Replace the hand-written Rego constraints with the community `gatekeeper-library` constraint templates and compare coverage.

Key Concepts Reinforced

- OS-level CIS hardening is a prerequisite to, not a substitute for, Kubernetes-level controls — this lab does both, deliberately
- The difference between **detective** controls (kube-bench reporting) and **preventive** controls (Gatekeeper admission rejection)
- Why “default deny” NetworkPolicy is the only network policy stance that’s actually zero trust — allow-lists only work if nothing is implicitly allowed first
- Proving a security control works means trying to violate it, not just deploying the control and assuming it’s effective

Appendix: Full Source Code

Every configuration and code file for this lab is listed below in full, organized by path, so the lab can be reproduced without needing the original repository.

ansible/roles/cis_hardening/handlers/main.yml

```
---
- name: restart sshd
  systemd:
    name: ssh
    state: restarted
```

ansible/roles/cis_hardening/tasks/main.yml

```
---
# A representative (not exhaustive) subset of CIS Ubuntu Linux Benchmark
# controls, chosen to be concrete and demonstrable within a lab, not a
# full compliance product.

- name: Ensure automatic security updates are enabled
  apt:
    name: unattended-upgrades
    state: present
    update_cache: true

- name: Disable core dumps (CIS 1.5.1)
  lineinfile:
    path: /etc/security/limits.conf
    line: "* hard core 0"
    create: true

- name: Ensure password expiration is configured (CIS 5.4.1.1)
  lineinfile:
    path: /etc/login.defs
    regexp: '^PASS_MAX_DAYS'
    line: 'PASS_MAX_DAYS 90'

- name: Ensure SSH root login is disabled (CIS 5.2.10)
  lineinfile:
    path: /etc/ssh/sshd_config
    regexp: '^#?PermitRootLogin'
    line: 'PermitRootLogin no'
  notify: restart sshd

- name: Ensure SSH password authentication is disabled (key-only) (CIS 5.2.4)
  lineinfile:
    path: /etc/ssh/sshd_config
    regexp: '^#?PasswordAuthentication'
    line: 'PasswordAuthentication no'
  notify: restart sshd

- name: Enable and configure auditd for kernel-level audit logging (CIS 4.1)
  apt:
```

```

    name: auditd
    state: present

- name: Ensure auditd service is running and enabled
  systemd:
    name: auditd
    state: started
    enabled: true

- name: Install k3s with audit logging enabled (Kubernetes-level CIS control)
  shell: |
    curl -sL https://get.k3s.io |
INSTALL_K3S_EXEC="--kube-apiserver-arg=audit-log-path=/var/log/k3s-audit.log
--kube-apiserver-arg=audit-log-maxage=30 --kube-apiserver-arg=audit-log-maxbackup=10" sh -
  args:
    creates: /usr/local/bin/k3s

- name: Wait for node Ready
  shell: k3s kubectl get nodes | grep -q Ready
  register: k3s_ready
  until: k3s_ready.rc == 0
  retries: 12
  delay: 10
  changed_when: false

- name: Set up kubeconfig for ubuntu user
  shell: |
    mkdir -p /home/ubuntu/.kube
    cp /etc/rancher/k3s/k3s.yaml /home/ubuntu/.kube/config
    chown ubuntu:ubuntu /home/ubuntu/.kube/config
  args:
    creates: /home/ubuntu/.kube/config

- name: Enable Kubernetes Pod Security Admission at the "restricted" level for the workloads namespace
  become_user: ubuntu
  environment: { KUBECONFIG: /home/ubuntu/.kube/config }
  shell: |
    k3s kubectl create namespace workloads --dry-run=client -o yaml | \
    k3s kubectl label --local -f - \
      pod-security.kubernetes.io/enforce=restricted \
      pod-security.kubernetes.io/audit=restricted \
      pod-security.kubernetes.io/warn=restricted \
    -o yaml --dry-run=client | \
    k3s kubectl apply -f -

```

ansible/roles/kube_bench/tasks/main.yml

```

---
- name: Download kube-bench
  unarchive:
    src: "https://github.com/aquasecurity/kube-bench/releases/download/v0.9.3/kube-
bench_0.9.3_linux_amd64.tar.gz"
    dest: /usr/local/bin
    remote_src: true

```

```

    creates: /usr/local/bin/kube-bench

- name: Run kube-bench and save the report (baseline BEFORE further remediation)
  shell: kube-bench run --targets node,policies > /home/ubuntu/kube-bench-baseline.txt 2>&1
  changed_when: false
  failed_when: false

- name: Fetch the kube-bench report to the control node
  fetch:
    src: /home/ubuntu/kube-bench-baseline.txt
    dest: "{{ playbook_dir }}/../generated/kube-bench-baseline.txt"
    flat: true

```

ansible/site.yml

```

---
- name: CIS OS hardening, k3s bootstrap, and kube-bench baseline
  hosts: hardened
  become: true
  roles:
    - cis_hardening
    - kube_bench

```

k8s/gatekeeper/constraint-no-privileged.yaml

```

apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sDenyPrivileged
metadata:
  name: deny-privileged-containers
spec:
  match:
    kinds:
      - apiGroups: [""]
        kinds: ["Pod"]
      namespaces: ["workloads"]

```

k8s/gatekeeper/constraint-require-limits.yaml

```

apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sRequireResourceLimits
metadata:
  name: require-resource-limits
spec:
  match:
    kinds:
      - apiGroups: [""]
        kinds: ["Pod"]
      namespaces: ["workloads"]

```

k8s/gatekeeper/constraint-template-no-privileged.yaml

```

apiVersion: templates.gatekeeper.sh/v1
kind: ConstraintTemplate
metadata:
  name: k8sdenyprivileged
spec:
  crd:
    spec:
      names:
        kind: K8sDenyPrivileged
  targets:
    - target: admission.k8s.gatekeeper.sh
      rego: |
        package k8sdenyprivileged

        violation[{"msg": msg}] {
          c := input.review.object.spec.containers[_]
          c.securityContext.privileged == true
          msg := sprintf("Privileged containers are not allowed: %v", [c.name])
        }

```

k8s/gatekeeper/constraint-template-require-limits.yaml

```

apiVersion: templates.gatekeeper.sh/v1
kind: ConstraintTemplate
metadata:
  name: k8srequireresourcelimits
spec:
  crd:
    spec:
      names:
        kind: K8sRequireResourceLimits
  targets:
    - target: admission.k8s.gatekeeper.sh
      rego: |
        package k8srequireresourcelimits

        violation[{"msg": msg}] {
          c := input.review.object.spec.containers[_]
          not c.resources.limits.cpu
          msg := sprintf("Container %v must set a CPU limit", [c.name])
        }

        violation[{"msg": msg}] {
          c := input.review.object.spec.containers[_]
          not c.resources.limits.memory
          msg := sprintf("Container %v must set a memory limit", [c.name])
        }

```

k8s/gatekeeper/install-note.md

Install OPA Gatekeeper itself (not a lab-authored manifest -- use the upstream release directly) before applying the constraint templates below:

```
kubectl apply -f https://raw.githubusercontent.com/open-policy-agent/gatekeeper/v3.16.0/deploy/gatekeeper.yaml
kubectl -n gatekeeper-system rollout status deployment/gatekeeper-controller-manager --timeout=180s
```

k8s/policies/allow-app-to-db.yaml

```
# Explicit allow: only pods labeled app=frontend may reach pods labeled
# app=backend-db, and only on port 5432. Everything else stays denied by
# the default-deny policy above.
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-to-db
  namespace: workloads
spec:
  podSelector:
    matchLabels:
      app: backend-db
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: frontend
      ports:
        - protocol: TCP
          port: 5432
```

k8s/policies/default-deny-network-policy.yaml

```
# Zero Trust networking baseline: deny ALL ingress/egress by default in the
# workloads namespace. Nothing can talk to anything unless explicitly
# allowed by a subsequent NetworkPolicy (see allow-app-to-db.yaml).
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
  namespace: workloads
spec:
  podSelector: {}
  policyTypes:
    - Ingress
    - Egress
```

k8s/policies/violating-pod-example.yaml

```
# This pod violates BOTH Gatekeeper constraints above (privileged=true,
# no resource limits) -- apply it after Gatekeeper is enforcing and
# confirm the API server rejects it with a clear admission-webhook error.
apiVersion: v1
kind: Pod
```

```

metadata:
  name: should-be-rejected
  namespace: workloads
spec:
  containers:
    - name: bad-container
      image: nginx
      securityContext:
        privileged: true

```

k8s/rbac/least-privilege-role.yaml

```

# A ServiceAccount scoped to read-only access on Pods/Services within its
# own namespace only -- the opposite of the default "cluster-admin
# ServiceAccount token mounted everywhere" anti-pattern.
apiVersion: v1
kind: ServiceAccount
metadata:
  name: app-readonly-sa
  namespace: workloads
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-service-reader
  namespace: workloads
rules:
  - apiGroups: [""]
    resources: ["pods", "services"]
    verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: app-readonly-binding
  namespace: workloads
subjects:
  - kind: ServiceAccount
    name: app-readonly-sa
    namespace: workloads
roleRef:
  kind: Role
  name: pod-service-reader
  apiGroup: rbac.authorization.k8s.io

```

terraform/main.tf

```

terraform {
  required_version = ">= 1.5.0"
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
    local = { source = "hashicorp/local", version = "~> 2.4" }
  }
}

```

```

}

provider "aws" { region = var.aws_region }

data "aws_vpc" "default" { default = true }

data "aws_subnets" "default" {
  filter {
    name   = "vpc-id"
    values = [data.aws_vpc.default.id]
  }
}

data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]
  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"]
  }
}

# Deliberately minimal security group -- this lab is ABOUT hardening,
# so we don't pre-open anything beyond what's needed to reach the node.
resource "aws_security_group" "hardened" {
  name_prefix = "lab9-hardened-sg-"
  vpc_id      = data.aws_vpc.default.id

  ingress { description = "SSH"    from_port = 22 to_port = 22 protocol = "tcp" cidr_blocks = [var.admin_cidr] }
  ingress { description = "K8s API" from_port = 6443 to_port = 6443 protocol = "tcp" cidr_blocks = [var.admin_cidr] }
  egress   { from_port = 0 to_port = 0 protocol = "-1" cidr_blocks = ["0.0.0.0/0"] }

  tags = { Name = "lab9-hardened-sg" }
}

resource "aws_instance" "hardened_node" {
  ami                = data.aws_ami.ubuntu.id
  instance_type      = var.instance_type
  subnet_id          = data.aws_subnets.default.ids[0]
  vpc_security_group_ids = [aws_security_group.hardened.id]
  key_name           = var.key_name
  root_block_device { volume_size = 20 }
  tags = { Name = "lab9-hardened-node" }
}

resource "local_file" "inventory" {
  filename = "${path.module}/../ansible/inventory.ini"
  content  = <<-EOT
[hardened]
${aws_instance.hardened_node.public_ip} ansible_user=ubuntu ansible_ssh_private_key_file=${var.private_key_path} ansible_
o StrictHostKeyChecking=no'
EOT
}

```

terraform/outputs.tf

```
output "hardened_node_public_ip" {  
    value = aws_instance.hardened_node.public_ip  
}
```

terraform/variables.tf

```
variable "aws_region"      { default = "us-east-1" }  
variable "instance_type"   { default = "t3.medium" }  
variable "key_name"        { type = string }  
variable "private_key_path" { type = string }  
variable "admin_cidr" {  
    description = "Restrict SSH/API access to your own IP, e.g. 1.2.3.4/32"  
    type        = string  
}
```

Lab 10 — HashiCorp Vault Dynamic Secrets for Kubernetes

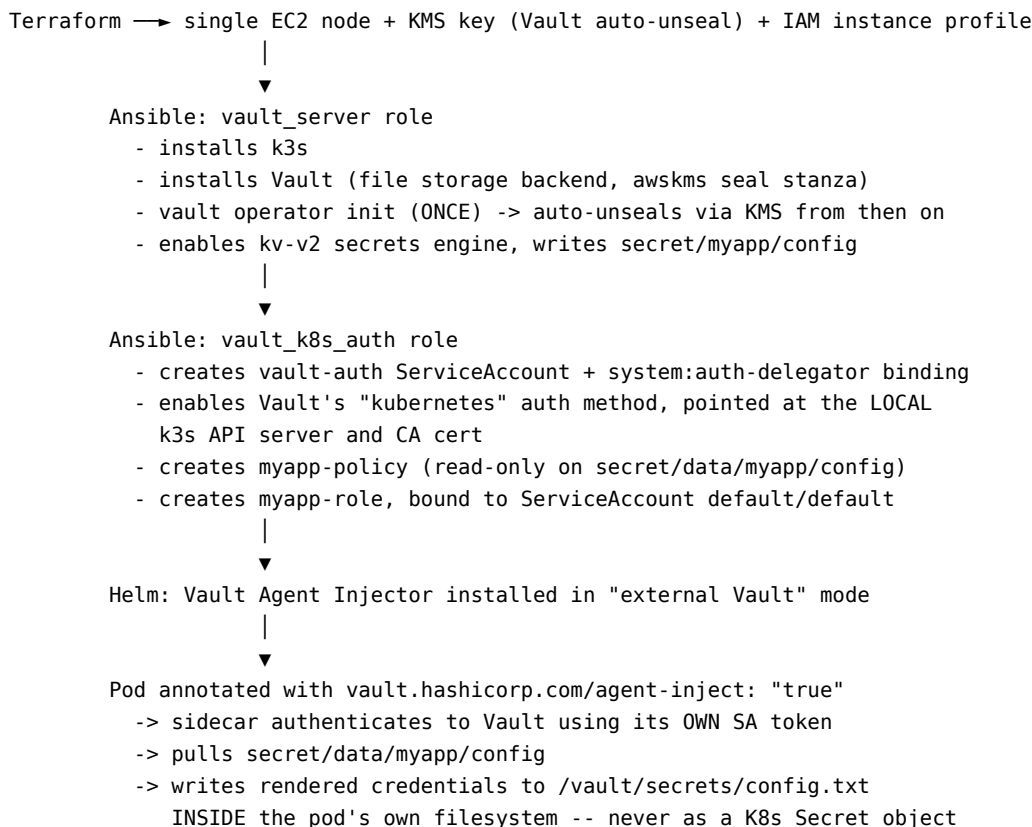
Difficulty: Advanced / Experimental

Builds on Labs 1-6 and Lab 9. Introduces a new tool (Vault) and a new authentication pattern (Kubernetes ServiceAccount -> Vault auth) that students have not seen in earlier labs.

Use Case

A security team wants to eliminate static secrets stored as Kubernetes Secret objects (which sit base64-encoded, not encrypted, in etcd by default) and instead have pods fetch credentials **live, at startup, directly from Vault**, authenticated purely by their Kubernetes identity — no static Vault token baked into any manifest. This lab provisions a node running both Vault (systemd service, AWS KMS auto-unseal) and a k3s cluster, wires Vault's Kubernetes auth method to trust that cluster, installs the Vault Agent Injector, and deploys a pod that receives database credentials via an auto-injected sidecar — with **no Kubernetes Secret object involved anywhere in the final workload**.

Architecture



Directory Layout

```

lab10-vault-dynamic-secrets/
├─ terraform/           # node + KMS key + IAM instance profile for auto-unseal
├─ ansible/
│   └─ site.yml
│   └─ generated/vault-init.json  # root token + recovery key fetched here (LAB ONLY)
└─ roles/
  
```

```

├── vault_server/{tasks,templates}/ # Vault install + init + kv secret
├── vault_k8s_auth/tasks/main.yml   # k8s auth method wiring
└── k8s/
    ├── vault-agent-injector-values.yaml # Helm values, external-Vault mode
    └── sample-app-with-injection.yaml    # pod with injected secret, zero K8s Secrets

```

Prerequisites

- Terraform >= 1.5.0, AWS CLI configured (KMS + IAM permissions)
- Ansible >= 2.14
- Helm >= 3.x installed on the lab node (or locally with kubeconfig copied down)
- An EC2 key pair

Step-by-Step Configuration & Execution

Step 1 — Provision the node, KMS key, and IAM instance profile

```

cd terraform
terraform init
terraform apply -var="key_name=my-ec2-keypair" -var="private_key_path=$HOME/.ssh/my-ec2-keypair.pem"

```

Step 2 — Bootstrap k3s, Vault (auto-unseal), and the Kubernetes auth method

```

cd ../ansible
until ansible -i inventory.ini vault_and_k8s -m ping; do sleep 5; done
ansible-playbook -i inventory.ini site.yml

```

Important: ansible/generated/vault-init.json now contains the Vault **root token** in plaintext. This is acceptable only because it's a disposable lab environment — never do this in production. Treat this file as sensitive and delete it during cleanup.

Step 3 — Confirm Vault auto-unsealed successfully

```

ssh -i ~/.ssh/my-ec2-keypair.pem ubuntu@<vault_node_public_ip>
export VAULT_ADDR=http://127.0.0.1:8200
vault status

```

Sealed must read false — this proves the KMS auto-unseal actually worked, without you ever running vault operator unseal manually.

Step 4 — Install the Vault Agent Injector via Helm

```

helm repo add hashicorp https://helm.releases.hashicorp.com
helm repo update
# Edit k8s/vault-agent-injector-values.yaml: replace VAULT_NODE_PRIVATE_IP
# with this node's actual private IP (`hostname -I`), then:
export KUBECONFIG=/home/ubuntu/.kube/config
helm install vault-injector hashicorp/vault -f vault-agent-injector-values.yaml -n vault-injector
--create-namespace
kubectl -n vault-injector rollout status deployment/vault-injector-agent-injector --timeout=180s

```

Step 5 — Deploy the sample app and watch secret injection happen

```
kubectl apply -f sample-app-with-injection.yaml
kubectl wait --for=condition=Ready pod/myapp-with-vault-secret --timeout=120s
```

Validation Steps

1. **No Kubernetes Secret object exists for this app's credentials** — this is the entire point of the lab:

```
kubectl get secrets | grep myapp
# Expected: no results
```

2. **The credentials nonetheless landed inside the running pod**, injected live by the Vault Agent sidecar:

```
kubectl exec myapp-with-vault-secret -c myapp -- cat /vault/secrets/config.txt
```

Output should show DB_HOST, DB_USER, DB_PASSWORD matching what was written with vault kv put in Step 2.

3. **The pod authenticated using its own ServiceAccount identity**, not a shared static token — confirm by checking the Vault Agent init container logs:

```
kubectl logs myapp-with-vault-secret -c vault-agent-init
```

Look for a successful auth/kubernetes/login call.

4. **Revoke access and confirm it actually blocks** — remove the ServiceAccount binding from the Vault role and confirm a NEW pod fails to fetch the secret:

```
vault write auth/kubernetes/role/myapp-role bound_service_account_names=nonexistent-sa
kubectl delete pod myapp-with-vault-secret
kubectl apply -f sample-app-with-injection.yaml
kubectl logs myapp-with-vault-secret -c vault-agent-init # should show a permission/login error
```

Restore the correct bound_service_account_names=default afterward.

5. **Auto-unseal survives a Vault restart** (proves KMS integration, not just first-boot luck):

```
sudo systemctl restart vault
sleep 5
vault status # Sealed: false, with no manual unseal command run
```

Cleanup

```
helm uninstall vault-injector -n vault-injector
cd terraform
terraform destroy -var="key_name=my-ec2-keypair" -var="private_key_path=$HOME/.ssh/my-ec2-keypair.pem"
rm -f ansible/generated/vault-init.json # contains a real root token -- do not keep this file around
```

Stretch Goals

- Replace the static kv-v2 secret with a **dynamic database secrets engine** (Vault generates a unique, time-limited DB credential per request instead of returning the same static value every time).

- Add a second Vault policy/role scoped to a different namespace and confirm cross-namespace access is correctly denied.
- Rotate the KMS key and confirm Vault continues to unseal correctly (or document what breaks and why).

Key Concepts Reinforced

- Why KMS auto-unseal is used in real deployments instead of the manual Shamir's-secret-sharing unseal ceremony
- Kubernetes ServiceAccount tokens as a Vault authentication mechanism — identity-based secret access instead of a shared static credential
- The Vault Agent Injector sidecar pattern: secrets rendered to an in-pod file, never stored as a Kubernetes API object
- How to prove a “least privilege” secret-access binding actually blocks access when the ServiceAccount doesn't match, not just that it grants access when it does

Appendix: Full Source Code

Every configuration and code file for this lab is listed below in full, organized by path, so the lab can be reproduced without needing the original repository.

ansible/roles/vault_k8s_auth/tasks/main.yml

```
---
# Wires Vault's Kubernetes auth method to the LOCAL k3s cluster running on
# this same node, so pods can authenticate to Vault using their own
# ServiceAccount token instead of any static credential.

- name: Extract the root token again for this role
  shell: cat /home/ubuntu/vault-init.json | python3 -c "import json,sys;
print(json.load(sys.stdin)['root_token'])"
  register: vault_root_token
  changed_when: false

- name: Create a ServiceAccount for Vault's token-review calls
  become_user: ubuntu
  environment: { KUBECONFIG: /home/ubuntu/.kube/config }
  shell: |
    k3s kubectl create serviceaccount vault-auth -n default --dry-run=client -o yaml | k3s kubectl apply
-f -
  changed_when: false

- name: Bind vault-auth to system:auth-delegator (required for Vault's TokenReview calls)
  become_user: ubuntu
  environment: { KUBECONFIG: /home/ubuntu/.kube/config }
  shell: |
    k3s kubectl create clusterrolebinding vault-auth-delegator \
      --clusterrole=system:auth-delegator \
      --serviceaccount=default:vault-auth \
      --dry-run=client -o yaml | k3s kubectl apply -f -
  changed_when: false

- name: Create a long-lived token Secret for the vault-auth ServiceAccount (K8s 1.24+ requires this
explicitly)
  become_user: ubuntu
  environment: { KUBECONFIG: /home/ubuntu/.kube/config }
  copy:
    dest: /home/ubuntu/vault-auth-secret.yaml
    content: |
      apiVersion: v1
      kind: Secret
      metadata:
        name: vault-auth-token
        namespace: default
        annotations:
          kubernetes.io/service-account.name: vault-auth
      type: kubernetes.io/service-account-token

- name: Apply the vault-auth token Secret
  become_user: ubuntu
  environment: { KUBECONFIG: /home/ubuntu/.kube/config }
```

```

command: k3s kubectl apply -f /home/ubuntu/vault-auth-secret.yaml
changed_when: false

- name: Read the reviewer JWT from the Secret
  become_user: ubuntu
  environment: { KUBECONFIG: /home/ubuntu/.kube/config }
  shell: k3s kubectl get secret vault-auth-token -n default -o jsonpath="{.data.token}" | base64 -d
  register: reviewer_jwt
  changed_when: false

- name: Read the cluster CA certificate
  slurp:
    src: /var/lib/rancher/k3s/server/tls/server-ca.crt
  register: k3s_ca_cert

- name: Enable the Kubernetes auth method in Vault
  environment:
    VAULT_ADDR: "http://127.0.0.1:8200"
    VAULT_TOKEN: "{{ vault_root_token.stdout }}"
  command: vault auth enable kubernetes
  register: auth_enable
  failed_when: "auth_enable.rc != 0 and 'path is already in use' not in auth_enable.stderr"
  changed_when: "auth_enable.rc == 0"

- name: Configure the Kubernetes auth method to trust THIS cluster
  environment:
    VAULT_ADDR: "http://127.0.0.1:8200"
    VAULT_TOKEN: "{{ vault_root_token.stdout }}"
  shell: |
    vault write auth/kubernetes/config \
      token_reviewer_jwt="{{ reviewer_jwt.stdout }}" \
      kubernetes_host="https://127.0.0.1:6443" \
      kubernetes_ca_cert="{{ k3s_ca_cert.content | b64decode }}" \
      disable_local_ca_jwt=true

- name: Write the Vault ACL policy granting read on the app's secret path
  copy:
    dest: /home/ubuntu/myapp-policy.hcl
    content: |
      path "secret/data/myapp/config" {
        capabilities = ["read"]
      }

- name: Create the Vault ACL policy
  environment:
    VAULT_ADDR: "http://127.0.0.1:8200"
    VAULT_TOKEN: "{{ vault_root_token.stdout }}"
  command: vault policy write myapp-policy /home/ubuntu/myapp-policy.hcl

- name: Create the Vault Kubernetes auth role binding ServiceAccount "default/default" to the policy
  environment:
    VAULT_ADDR: "http://127.0.0.1:8200"
    VAULT_TOKEN: "{{ vault_root_token.stdout }}"
  command: >
    vault write auth/kubernetes/role/myapp-role

```

```
bound_service_account_names=default
bound_service_account_namespaces=default
policies=myapp-policy
ttl=1h
```

ansible/roles/vault_server/tasks/main.yml

```
---
- name: Install k3s (Vault and Kubernetes run on the same lab node)
  shell: curl -sL https://get.k3s.io | sh -
  args:
    creates: /usr/local/bin/k3s

- name: Wait for node Ready
  shell: k3s kubectl get nodes | grep -q Ready
  register: k3s_ready
  until: k3s_ready.rc == 0
  retries: 12
  delay: 10
  changed_when: false

- name: Set up kubeconfig for ubuntu user
  shell: |
    mkdir -p /home/ubuntu/.kube
    cp /etc/rancher/k3s/k3s.yaml /home/ubuntu/.kube/config
    chown ubuntu:ubuntu /home/ubuntu/.kube/config
  args:
    creates: /home/ubuntu/.kube/config

- name: Add the HashiCorp apt GPG key
  get_url:
    url: https://apt.releases.hashicorp.com/gpg
    dest: /etc/apt/keyrings/hashicorp.asc
    mode: "0644"

- name: Add the HashiCorp apt repository
  apt_repository:
    repo: "deb [signed-by=/etc/apt/keyrings/hashicorp.asc] https://apt.releases.hashicorp.com {{
ansible_distribution_release }}"
    state: present

- name: Install Vault
  apt:
    name: vault
    state: present
    update_cache: true

- name: Create Vault data directory
  file:
    path: /opt/vault/data
    state: directory
    owner: vault
    group: vault
```

```
- name: Write Vault server configuration (file storage + AWS KMS auto-unseal)
  template:
    src: vault.hcl.j2
    dest: /etc/vault.d/vault.hcl
    mode: "0640"
    owner: vault
    group: vault

- name: Start and enable the Vault service
  systemd:
    name: vault
    state: started
    enabled: true

- name: Wait for the Vault API to respond
  uri:
    url: "http://127.0.0.1:8200/v1/sys/health"
    status_code: [200, 429, 472, 473, 501, 503]
    register: vault_health
    until: vault_health.status is defined
    retries: 12
    delay: 5

- name: Check whether Vault is already initialized
  environment: { VAULT_ADDR: "http://127.0.0.1:8200" }
  command: vault status -format=json
  register: vault_status_raw
  changed_when: false
  failed_when: false

- name: Initialize Vault (only runs once; auto-unseals via KMS thereafter)
  environment: { VAULT_ADDR: "http://127.0.0.1:8200" }
  shell: vault operator init -key-shares=1 -key-threshold=1 -format=json > /home/ubuntu/vault-init.json
  args:
    creates: /home/ubuntu/vault-init.json
  when: "'\"initialized\": false' in vault_status_raw.stdout or vault_status_raw.rc != 0"

- name: Fetch the Vault init output (root token + recovery key) to the control node
  fetch:
    src: /home/ubuntu/vault-init.json
    dest: "{{ playbook_dir }}/generated/vault-init.json"
    flat: true
    ignore_errors: true

- name: Extract the root token for use in subsequent tasks
  shell: cat /home/ubuntu/vault-init.json | python3 -c "import json,sys;
print(json.load(sys.stdin)['root_token'])"
  register: vault_root_token
  changed_when: false

- name: Enable the kv-v2 secrets engine at path secret/
  environment:
    VAULT_ADDR: "http://127.0.0.1:8200"
    VAULT_TOKEN: "{{ vault_root_token.stdout }}"
  command: vault secrets enable -path=secret -version=2 kv
```

```

register: kv_enable
failed_when: "kv_enable.rc != 0 and 'path is already in use' not in kv_enable.stderr"
changed_when: "kv_enable.rc == 0"

- name: Write a sample application secret (this is what the injected pod will read)
  environment:
    VAULT_ADDR: "http://127.0.0.1:8200"
    VAULT_TOKEN: "{{ vault_root_token.stdout }}"
  command: >
    vault kv put secret/myapp/config
    username=app_service_account
    password=S3cretDBPassw0rd!
    db_host=internal-db.example.com

```

ansible/roles/vault_server/templates/vault.hcl.j2

```

storage "file" {
  path = "/opt/vault/data"
}

listener "tcp" {
  address      = "0.0.0.0:8200"
  tls_disable = 1  # lab only -- always enable TLS in real deployments
}

seal "awskms" {
  region      = "{{ aws_region }}"
  kms_key_id = "{{ kms_key_id }}"
}

api_addr      = "http://0.0.0.0:8200"
ui            = true
disable_mlock = true

```

ansible/site.yml

```

---
- name: Bootstrap k3s + Vault (KMS auto-unseal) + Kubernetes auth method
  hosts: vault_and_k8s
  become: true
  roles:
    - vault_server
    - vault_k8s_auth

```

k8s/sample-app-with-injection.yaml

```

# A pod with NO Kubernetes Secret object anywhere -- the database
# credentials are pulled live from Vault by the injected sidecar at
# startup and written to a file inside the pod's own filesystem, never
# touching etcd.
apiVersion: v1

```

```

kind: Pod
metadata:
  name: myapp-with-vault-secret
  namespace: default
  annotations:
    vault.hashicorp.com/agent-inject: "true"
    vault.hashicorp.com/role: "myapp-role"
    vault.hashicorp.com/agent-inject-secret-config.txt: "secret/data/myapp/config"
    vault.hashicorp.com/agent-inject-template-config.txt: |
      {{- with secret "secret/data/myapp/config" -}}
      DB_HOST={{ .Data.data.db_host }}
      DB_USER={{ .Data.data.username }}
      DB_PASSWORD={{ .Data.data.password }}
      {{- end -}}
spec:
  serviceAccountName: default
  containers:
    - name: myapp
      image: busybox
      command:
        - sh
        - -c
        - "sleep infinity"

```

k8s/vault-agent-injector-values.yaml

```

# Helm values for the official HashiCorp vault-helm chart, configured in
# "external Vault" mode: only the Agent Injector runs inside Kubernetes;
# the actual Vault server is the systemd service running on the host
# (installed by the vault_server Ansible role), NOT a pod in this cluster.
injector:
  enabled: true
  externalVaultAddr: "http://VAULT_NODE_PRIVATE_IP:8200"

server:
  enabled: false

```

terraform/main.tf

```

terraform {
  required_version = ">= 1.5.0"
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
    local = { source = "hashicorp/local", version = "~> 2.4" }
  }
}

provider "aws" { region = var.aws_region }

data "aws_vpc" "default" { default = true }

data "aws_subnets" "default" {
  filter {

```

```

    name    = "vpc-id"
    values = [data.aws_vpc.default.id]
  }
}

data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]
  filter {
    name     = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"]
  }
}

# KMS key used for Vault auto-unseal -- avoids the manual "5 unseal keys"
# ceremony for this lab, matching how most real deployments do it.
resource "aws_kms_key" "vault_unseal" {
  description          = "Vault auto-unseal key for lab10"
  deletion_window_in_days = 7
}

resource "aws_iam_role" "vault_node_role" {
  name = "lab10-vault-node-role"
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect    = "Allow"
      Principal = { Service = "ec2.amazonaws.com" }
      Action    = "sts:AssumeRole"
    }]
  })
}

resource "aws_iam_role_policy" "vault_kms" {
  name = "vault-kms-unseal-policy"
  role = aws_iam_role.vault_node_role.id
  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect    = "Allow"
      Action    = ["kms:Encrypt", "kms:Decrypt", "kms:DescribeKey"]
      Resource = aws_kms_key.vault_unseal.arn
    }]
  })
}

resource "aws_iam_instance_profile" "vault_node" {
  name = "lab10-vault-node-profile"
  role = aws_iam_role.vault_node_role.name
}

resource "aws_security_group" "vault" {
  name_prefix = "lab10-vault-sg-"
  vpc_id      = data.aws_vpc.default.id

  ingress { description = "SSH"      from_port = 22  to_port = 22  protocol = "tcp" cidr_blocks = ["0.0.0.0/0"] }

```

```

    ingress { description = "Vault API" from_port = 8200 to_port = 8200 protocol = "tcp" cidr_blocks = ["0.0.0.0/0"] }
    ingress { description = "K8s API"   from_port = 6443 to_port = 6443 protocol = "tcp" cidr_blocks = ["0.0.0.0/0"] }
    ingress { description = "NodePort"  from_port = 30000 to_port = 32767 protocol = "tcp" cidr_blocks = ["0.0.0.0/0"] }
    egress  { from_port = 0 to_port = 0 protocol = "-1" cidr_blocks = ["0.0.0.0/0"] }

    tags = { Name = "lab10-vault-sg" }
}

resource "aws_instance" "vault_node" {
    ami                = data.aws_ami.ubuntu.id
    instance_type      = var.instance_type
    subnet_id          = data.aws_subnets.default.ids[0]
    vpc_security_group_ids = [aws_security_group.vault.id]
    key_name            = var.key_name
    iam_instance_profile = aws_iam_instance_profile.vault_node.name
    root_block_device { volume_size = 20 }
    tags = { Name = "lab10-vault-k8s-node" }
}

resource "local_file" "inventory" {
    filename = "${path.module}/../ansible/inventory.ini"
    content  = <<-EOT
[vault_and_k8s]
${aws_instance.vault_node.public_ip} ansible_user=ubuntu ansible_ssh_private_key_file=${var.private_key_path} ansible_
o StrictHostKeyChecking=no'

[vault_and_k8s:vars]
kms_key_id=${aws_kms_key.vault_unseal.key_id}
aws_region=${var.aws_region}
EOT
}

```

terraform/outputs.tf

```

output "vault_node_public_ip" {
    value = aws_instance.vault_node.public_ip
}

output "kms_key_id" {
    value = aws_kms_key.vault_unseal.key_id
}

```

terraform/variables.tf

```

variable "aws_region"      { default = "us-east-1" }
variable "instance_type"   { default = "t3.medium" }
variable "key_name"        { type = string }
variable "private_key_path" { type = string }

```

Lab 11 — Autoscaling Under Chaos (HPA + KEDA + Chaos Mesh)

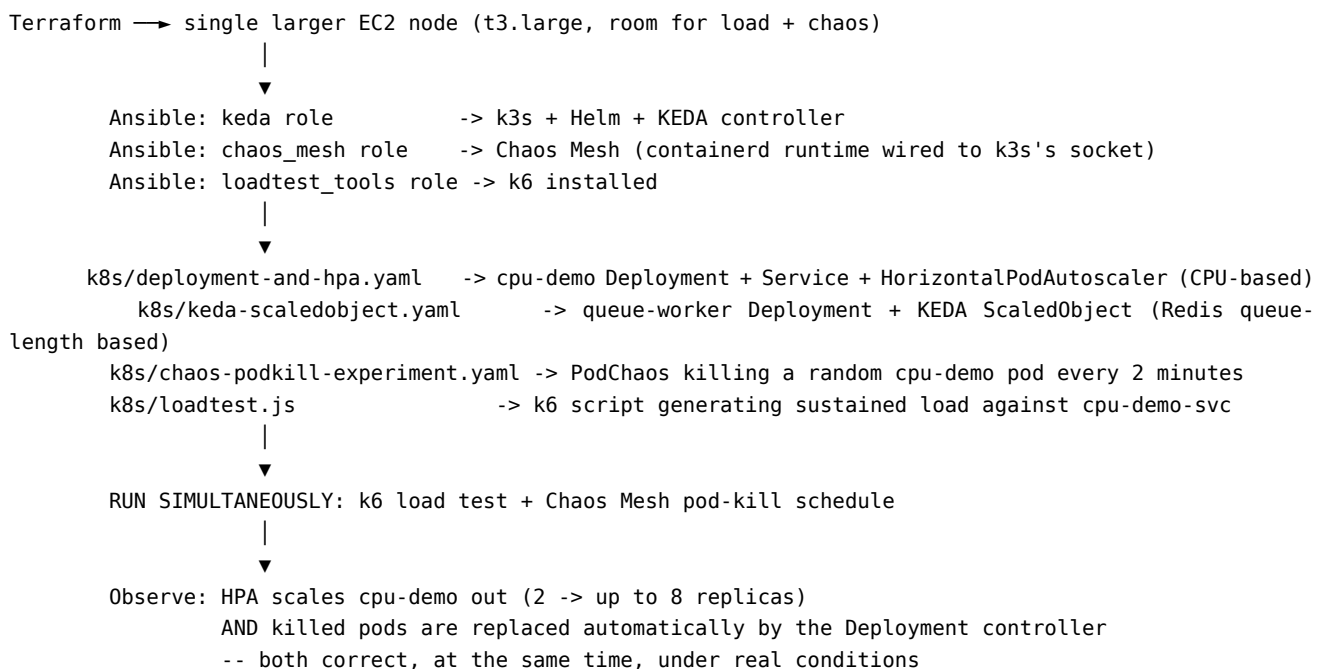
Difficulty: Advanced / Experimental

Builds on Labs 1-6. This is the “prove it survives” capstone lab: students generate real load AND real failures simultaneously, then verify the system recovers on its own.

Use Case

A platform team needs to prove a workload can **both scale under load and recover from random pod failures at the same time** — not as two separate tests, but concurrently, the way real production incidents actually happen (a traffic spike during a partial outage). This lab provisions a single larger node, installs KEDA (event-driven autoscaling beyond plain CPU-based HPA) and Chaos Mesh (fault injection), then runs a sustained k6 load test against a CPU-bound deployment while Chaos Mesh kills a random pod every two minutes — validating that HPA scales out under load AND the killed pod is automatically replaced, without any manual intervention, all in the same time window.

Architecture



Directory Layout

```

lab11-autoscaling-chaos/
├─ terraform/          # single t3.large node
├─ ansible/
│   ├─ site.yml
│   └─ roles/
│       ├─ keda/tasks/main.yml          # k3s + Helm + KEDA
│       ├─ chaos_mesh/tasks/main.yml    # Chaos Mesh, wired to k3s's containerd socket
│       └─ loadtest_tools/tasks/main.yml # k6 install
└─ k8s/
    └─ deployment-and-hpa.yaml          # CPU-based HPA demo
  
```

```
|— keda-scaledobject.yaml      # event-driven (Redis queue) autoscaling demo
|— chaos-podkill-experiment.yaml # scheduled pod-kill chaos experiment
|— loadtest.js                # k6 script
```

Prerequisites

- Terraform $\geq$ 1.5.0, AWS CLI configured
- Ansible $\geq$ 2.14
- An EC2 key pair
- (Optional, for the KEDA queue demo) a Redis instance reachable from the cluster — simplest is `k3s kubectl run redis --image=redis --port=6379 --expose`

Step-by-Step Configuration & Execution

Step 1 — Provision the node

```
cd terraform
terraform init
terraform apply -var="key_name=my-ec2-keypair" -var="private_key_path=$HOME/.ssh/my-ec2-keypair.pem"
```

Step 2 — Bootstrap k3s, KEDA, Chaos Mesh, and k6

```
cd ../ansible
until ansible -i inventory.ini chaos -m ping; do sleep 5; done
ansible-playbook -i inventory.ini site.yml
```

Step 3 — Deploy the CPU-demo workload with its HPA

```
ssh -i ~/.ssh/my-ec2-keypair.pem ubuntu@<chaos_node_public_ip>
export KUBECONFIG=/home/ubuntu/.kube/config
# scp k8s/deployment-and-hpa.yaml up first, then:
k3s kubectl apply -f deployment-and-hpa.yaml
k3s kubectl get svc cpu-demo-svc      # note the assigned NodePort
```

Step 4 — Apply the Chaos Mesh pod-kill schedule (starts immediately, runs every 2 minutes)

```
k3s kubectl apply -f chaos-podkill-experiment.yaml
```

Step 5 — Edit and run the k6 load test, WHILE chaos is active

```
# edit loadtest.js: replace CPU_DEMO_NODE_IP:NODEPORT with the real values
k6 run loadtest.js
```

This runs for 5 minutes at 50 virtual users — long enough to observe multiple HPA scaling decisions and at least 2 scheduled pod kills.

Step 6 — (Optional) Try the KEDA event-driven demo

```
k3s kubectl run redis --image=redis --port=6379 --expose
k3s kubectl apply -f keda-scaledobject.yaml
# push items onto the queue and watch queue-worker scale from 0:
k3s kubectl exec -it redis-<pod-suffix> -- redis-cli rpush work_queue a b c d e f g h i j k l
k3s kubectl get deployment queue-worker --watch
```

Validation Steps

1. HPA actually scaled the deployment out during the load test

```
kubectl get hpa cpu-demo-hpa --watch
```

REPLICAS should climb from 2 toward higher values as TARGETS (CPU%) rises above 50% during the k6 run.

2. Chaos Mesh actually killed pods on schedule — confirm from events, not assumption

```
kubectl get events --field-selector reason=Killing -n default
kubectl describe podchaos kill-cpu-demo-pod
```

3. Killed pods were automatically replaced — cross-reference pod AGE during the test window; you should see some cpu-demo pods with an age noticeably younger than others, proving a restart happened mid-test without manual intervention:

```
kubectl get pods -l app=cpu-demo -o wide
```

4. No request-serving gap during a pod kill (the real point of combining load + chaos): check the k6 summary output at the end of the run for error rate / failed request count — a well-configured Service with multiple replicas should show negligible impact from a single pod kill.

5. KEDA scaled from zero (if you ran the optional Step 6):

```
kubectl get deployment queue-worker --watch
# replicas should go 0 -> N as items are pushed, then back toward 0
# after the queue drains and the cooldown period passes
```

6. Post-test report (write this up as part of the lab): plot or tabulate HPA replica count over time alongside the timestamps of each Chaos Mesh pod-kill event, from the k6 run's start to finish, and describe in your own words what would have happened without the HPA (manual scaling) and without Chaos Mesh validation (an untested assumption of resilience).

Cleanup

```
kubectl delete -f keda-scaledobject.yaml --ignore-not-found
kubectl delete -f chaos-podkill-experiment.yaml
kubectl delete -f deployment-and-hpa.yaml
cd terraform
terraform destroy -var="key_name=my-ec2-keypair" -var="private_key_path=$HOME/.ssh/my-ec2-keypair.pem"
```

Stretch Goals

- Add a second, more aggressive Chaos Mesh experiment (NetworkChaos with injected latency) running concurrently with the pod-kill schedule, and observe compounding effects.
- Replace the CPU-based HPA with a KEDA ScaledObject driven by a custom Prometheus query, and compare scaling responsiveness.

- Research and document how this single-node lab setup maps to a real Cluster Autoscaler / Karpenter configuration, where new EC2 **nodes** (not just pods) are added when the cluster itself runs out of capacity.

Key Concepts Reinforced

- The difference between CPU-based HPA and event-driven KEDA autoscaling
- Why chaos experiments must run **concurrently** with load tests to be a meaningful resilience test, not run as two separate, easier scenarios
- Confirming self-healing behavior from actual Kubernetes events, not from the absence of an error message
- Connecting a single-node lab's HPA behavior conceptually to real node-level autoscaling (Cluster Autoscaler/Karpenter) in production

Appendix: Full Source Code

Every configuration and code file for this lab is listed below in full, organized by path, so the lab can be reproduced without needing the original repository.

ansible/roles/chaos_mesh/tasks/main.yml

```
---
- name: Add the Chaos Mesh Helm repo
  become_user: ubuntu
  environment: { KUBECONFIG: /home/ubuntu/.kube/config }
  command: helm repo add chaos-mesh https://charts.chaos-mesh.org
  changed_when: false

- name: Update Helm repos
  become_user: ubuntu
  environment: { KUBECONFIG: /home/ubuntu/.kube/config }
  command: helm repo update
  changed_when: false

- name: Install Chaos Mesh (using containerd's local socket, matching k3s's runtime)
  become_user: ubuntu
  environment: { KUBECONFIG: /home/ubuntu/.kube/config }
  command: >
    helm install chaos-mesh chaos-mesh/chaos-mesh
    --namespace chaos-mesh --create-namespace
    --set chaosDaemon.runtime=containerd
    --set chaosDaemon.socketPath=/run/k3s/containerd/containerd.sock
  register: chaos_install
  failed_when: "chaos_install.rc != 0 and 'cannot re-use' not in chaos_install.stderr"
  changed_when: "chaos_install.rc == 0"
```

ansible/roles/keda/tasks/main.yml

```
---
- name: Install k3s with metrics-server enabled (needed for HPA CPU metrics)
  shell: curl -sL https://get.k3s.io | sh -
  args:
    creates: /usr/local/bin/k3s

- name: Wait for node Ready
  shell: k3s kubectl get nodes | grep -q Ready
  register: k3s_ready
  until: k3s_ready.rc == 0
  retries: 12
  delay: 10
  changed_when: false

- name: Set up kubeconfig for ubuntu user
  shell: |
    mkdir -p /home/ubuntu/.kube
    cp /etc/rancher/k3s/k3s.yaml /home/ubuntu/.kube/config
    chown ubuntu:ubuntu /home/ubuntu/.kube/config
  args:
```

```

    creates: /home/ubuntu/.kube/config

- name: Install Helm
  shell: curl -fsSL https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
  args:
    creates: /usr/local/bin/helm

- name: Add the KEDA Helm repo
  become_user: ubuntu
  environment: { KUBECONFIG: /home/ubuntu/.kube/config }
  command: helm repo add kedacore https://kedacore.github.io/charts
  changed_when: false

- name: Update Helm repos
  become_user: ubuntu
  environment: { KUBECONFIG: /home/ubuntu/.kube/config }
  command: helm repo update
  changed_when: false

- name: Install KEDA (event-driven autoscaling controller)
  become_user: ubuntu
  environment: { KUBECONFIG: /home/ubuntu/.kube/config }
  command: helm install keda kedacore/keda --namespace keda --create-namespace
  register: keda_install
  failed_when: "keda_install.rc != 0 and 'cannot re-use' not in keda_install.stderr"
  changed_when: "keda_install.rc == 0"

```

ansible/roles/loadtest_tools/tasks/main.yml

```

---
- name: Install k6 load-testing tool
  shell: |
    curl -s https://dl.k6.io/key.gpg | gpg --dearmor | tee /usr/share/keyrings/k6-archive-keyring.gpg > /dev/null
    echo "deb [signed-by=/usr/share/keyrings/k6-archive-keyring.gpg] https://dl.k6.io/deb stable main" | tee /etc/apt/sources.list.d/k6.list
    apt-get update
    apt-get install -y k6
  args:
    creates: /usr/bin/k6

```

ansible/site.yml

```

---
- name: Bootstrap k3s, KEDA, Chaos Mesh, and load-testing tools
  hosts: chaos
  become: true
  roles:
    - keda
    - chaos_mesh
    - loadtest_tools

```

k8s/chaos-podkill-experiment.yaml

```
# Chaos Mesh experiment: kill a random cpu-demo pod every 2 minutes for
# the duration of the test window, WHILE the load test and HPA are both
# running, to prove the system self-heals under simultaneous load + failure.
apiVersion: chaos-mesh.org/v1alpha1
kind: PodChaos
metadata:
  name: kill-cpu-demo-pod
  namespace: default
spec:
  action: pod-kill
  mode: one
  selector:
    namespaces: ["default"]
    labelSelectors:
      app: cpu-demo
  scheduler:
    cron: "@every 2m"
```

k8s/deployment-and-hpa.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: cpu-demo
  namespace: default
spec:
  replicas: 2
  selector:
    matchLabels:
      app: cpu-demo
  template:
    metadata:
      labels:
        app: cpu-demo
    spec:
      containers:
        - name: cpu-demo
          image: k8s.gcr.io/hpa-example
          ports:
            - containerPort: 80
          resources:
            requests: { cpu: "200m" }
            limits: { cpu: "500m" }
---
apiVersion: v1
kind: Service
metadata:
  name: cpu-demo-svc
  namespace: default
spec:
  selector:
    app: cpu-demo
```

```

ports:
  - port: 80
    type: NodePort
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: cpu-demo-hpa
  namespace: default
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: cpu-demo
  minReplicas: 2
  maxReplicas: 8
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 50

```

k8s/keda-scaledobject.yaml

```

# KEDA example: scale a worker deployment based on a Redis list length
# (an EVENT-DRIVEN metric), not just CPU -- something native HPA cannot
# do on its own. Requires a redis instance reachable at redis.default.svc.
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: queue-worker-scaledobject
  namespace: default
spec:
  scaleTargetRef:
    name: queue-worker
  minReplicaCount: 0
  maxReplicaCount: 10
  triggers:
    - type: redis
      metadata:
        address: redis.default.svc:6379
        listName: work_queue
        listLength: "5" # add 1 replica per 5 queued items
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: queue-worker
  namespace: default
spec:
  replicas: 0
  selector:

```

```
matchLabels:
  app: queue-worker
template:
  metadata:
    labels:
      app: queue-worker
  spec:
    containers:
      - name: worker
        image: busybox
        command: ["sh", "-c", "sleep infinity"]
        resources:
          requests: { cpu: "50m", memory: "32Mi" }
```

k8s/loadtest.js

```
// k6 load test script -- generates sustained CPU load against cpu-demo-svc
// so the HPA has a real signal to scale on.
import http from 'k6/http';
import { sleep } from 'k6';

export const options = {
  vus: 50,          // 50 virtual users
  duration: '5m',
};

export default function () {
  http.get('http://CPU_DEMO_NODE_IP:NODEPORT/');
  sleep(0.1);
}
```

terraform/main.tf

```
terraform {
  required_version = ">= 1.5.0"
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
    local = { source = "hashicorp/local", version = "~> 2.4" }
  }
}

provider "aws" { region = var.aws_region }

data "aws_vpc" "default" { default = true }

data "aws_subnets" "default" {
  filter {
    name = "vpc-id"
    values = [data.aws_vpc.default.id]
  }
}

data "aws_ami" "ubuntu" {
  most_recent = true
```

```

owners      = ["099720109477"]
filter {
  name      = "name"
  values    = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"]
}
}

resource "aws_security_group" "chaos" {
  name_prefix = "lab11-chaos-sg-"
  vpc_id      = data.aws_vpc.default.id

  ingress { description = "SSH"      from_port = 22   to_port = 22   protocol = "tcp" cidr_blocks = ["0.0.0.0/0"] }
  ingress { description = "K8s API"  from_port = 6443 to_port = 6443 protocol = "tcp" cidr_blocks = ["0.0.0.0/0"] }
  ingress { description = "NodePort" from_port = 30000 to_port = 32767 protocol = "tcp" cidr_blocks = ["0.0.0.0/0"] }
  egress   { from_port = 0 to_port = 0 protocol = "-1" cidr_blocks = ["0.0.0.0/0"] }

  tags = { Name = "lab11-chaos-sg" }
}

# NOTE: a single, larger node is used (instead of a multi-node autoscaling
# group) so students can run HPA/KEDA + Chaos Mesh experiments without
# needing a full cloud-provider Cluster Autoscaler integration. The README
# discusses how this maps to a real Cluster Autoscaler / Karpenter setup.
resource "aws_instance" "chaos_node" {
  ami                = data.aws_ami.ubuntu.id
  instance_type      = var.instance_type
  subnet_id          = data.aws_subnets.default.ids[0]
  vpc_security_group_ids = [aws_security_group.chaos.id]
  key_name            = var.key_name
  root_block_device { volume_size = 30 }
  tags = { Name = "lab11-chaos-node" }
}

resource "local_file" "inventory" {
  filename = "${path.module}/../ansible/inventory.ini"
  content  = <<-EOT
[chaos]
${aws_instance.chaos_node.public_ip} ansible_user=ubuntu ansible_ssh_private_key_file=${var.private_key_path} ansible_
o StrictHostKeyChecking=no'
EOT
}

```

terraform/outputs.tf

```

output "chaos_node_public_ip" {
  value = aws_instance.chaos_node.public_ip
}

```

terraform/variables.tf

```

variable "aws_region"      { default = "us-east-1" }
variable "instance_type"   { default = "t3.large" }
variable "key_name"        { type = string }
variable "private_key_path" { type = string }

```